

ROS 2 URDF Modeling of a 4-DOF Industrial Robotic Arm

Project Report

Submitted by
Ajay Bisht

Robo AI | My Equation
Tools: ROS 2 (Jazzy) · URDF / Xacro · RViz2 · robot_state_publisher

Contents

1. Problem Statement	3
2. Robot Structure Design	3
3. URDF Link Definition	4
4. Joint Definition	4
5. Assembly Validation	5
6. RViz Visualization	5
7. TF Tree Verification	5
8. Tools Used	6
9. File Organization	6
10. Results and Evaluation	7
11. Extension: Simulation and Pick-and-Place	7
12. Conclusion	9

1. Problem Statement

The objective of this project is to design and implement a ROS 2 URDF model of a 4-degree-of-freedom (4-DOF) industrial robotic arm. The model is required to define all links and joints correctly, include both visual and collision geometry, use appropriate joint types and limits, load successfully in RViz, and produce a correct transform (TF) tree.

The arm modelled here is based on the ROBOTIS OpenMANIPULATOR-X, a compact 4-DOF industrial-style manipulator with a two-finger gripper. Its real STL meshes are used for the visual and collision geometry so that the model is dimensionally accurate rather than built from primitive shapes.

2. Robot Structure Design

The manipulator is an open kinematic chain of five rigid links connected by four actuated revolute joints, followed by a two-finger prismatic gripper. Joint 1 provides base rotation (yaw) about the vertical Z axis; joints 2, 3 and 4 provide pitch rotation about the Y axis, together controlling the position of the end-effector in the vertical plane. A fixed world link anchors the base to the ground frame, and a fixed end-effector frame marks the tool centre point.

2.1 Links

Link	Role	Geometry source
world	Fixed ground/reference frame	(virtual, no mesh)
link1	Base of the arm	link1.stl
link2	Shoulder link (joint 1 output)	link2.stl
link3	Upper arm (joint 2 output)	link3.stl
link4	Forearm (joint 3 output)	link4.stl
link5	Wrist / gripper base (joint 4 output)	link5.stl
gripper_left_link	Left finger	gripper_left_palm.stl
gripper_right_link	Right finger (mimics left)	gripper_right_palm.stl
end_effector_link	Tool centre point frame	(marker)

2.2 Joints

The kinematic chain is summarised below. Angular limits are given in radians and prismatic limits in metres.

Joint	Type	Parent → Child	Axis	Limits
world_fixed	fixed	world → link1	–	–
joint1	revolute	link1 → link2	Z	-3.1416 – 3.1416
joint2	revolute	link2 → link3	Y	-1.5 – 1.5

Joint	Type	Parent → Child	Axis	Limits
joint3	revolute	link3 → link4	Y	-1.5 – 1.4
joint4	revolute	link4 → link5	Y	-1.7 – 1.97
gripper_left_joint	prismatic	link5 → gripper_left_link	Y	-0.011 – 0.02
gripper_right_joint	prismatic (mimic)	link5 → gripper_right_link	Y	-0.011 – 0.02
end_effector_joint	fixed	link5 → end_effector_link	–	–

3. URDF Link Definition

Each link is defined in URDF with three elements: a <visual> block for rendering, a <collision> block for collision checking, and an <inertial> block giving the mass and inertia tensor. Visual and collision geometry reference the same real STL mesh, scaled from millimetres to metres (scale 0.001). Inertial values are taken from the manufacturer's specification so that the model is physically consistent. A representative link definition is shown below.

```
<link name="link2">
  <visual>
    <origin rpy="0 0 0" xyz="0 0 0.019"/>
    <geometry>
      <mesh filename="package://omx_mujoco/mujoco/assets/link2.stl"
        scale="0.001 0.001 0.001"/>
    </geometry>
  </visual>
  <collision>
    <origin rpy="0 0 0" xyz="0 0 0.019"/>
    <geometry>
      <mesh filename="package://omx_mujoco/mujoco/assets/link2.stl"
        scale="0.001 0.001 0.001"/>
    </geometry>
  </collision>
  <inertial>
    <mass value="9.84e-02"/>
    <inertia ixx="3.45e-05" iyy="3.27e-05" izz="1.89e-05"
      ixy="0" ixz="0" iyz="0"/>
  </inertial>
</link>
```

4. Joint Definition

Joints connect a parent link to a child link and define the axis and range of motion. The four arm joints are revolute; joint 1 rotates about Z and joints 2–4 about Y. Each joint declares an <origin> (the fixed offset from the parent frame), an <axis>, and a <limit> giving the lower and upper bounds, effort and velocity. The two gripper joints are prismatic, and the right finger is defined with a <mimic> tag so that it follows the left finger, giving symmetric opening and closing from a single command.

```
<joint name="joint1" type="revolute">
  <parent link="link1"/>
  <child link="link2"/>
  <origin rpy="0 0 0" xyz="0.012 0 0"/>
  <axis xyz="0 0 1"/>
```

```
<limit effort="1000" velocity="4.8"  
      lower="-3.14159" upper="3.14159"/>  
</joint>
```

5. Assembly Validation

All links and joints are assembled into a single robot description authored as a Xacro file (omx.urdf.xacro), which expands to a complete URDF. The description was validated by loading it with `robot_state_publisher`, which parses the URDF, builds the kinematic tree, and publishes the resulting transforms. Successful parsing with no errors, together with a single connected kinematic chain from world to the end-effector, confirms that parent-child relationships, joint origins and axes are consistent.

- The Xacro expands without errors (checked with: `xacro omx.urdf.xacro`).
- `robot_state_publisher` loads the description and reports the model initialised.
- A single connected tree is formed — no orphaned or duplicated links.

6. RViz Visualization

The URDF was loaded and visualised with its real mesh geometry, with the fixed frame set to world. Moving the joints updates the model in real time, confirming correct visual geometry and joint behaviour. The figure below shows the arm rendered in the simulation environment with the workspace table and target object.

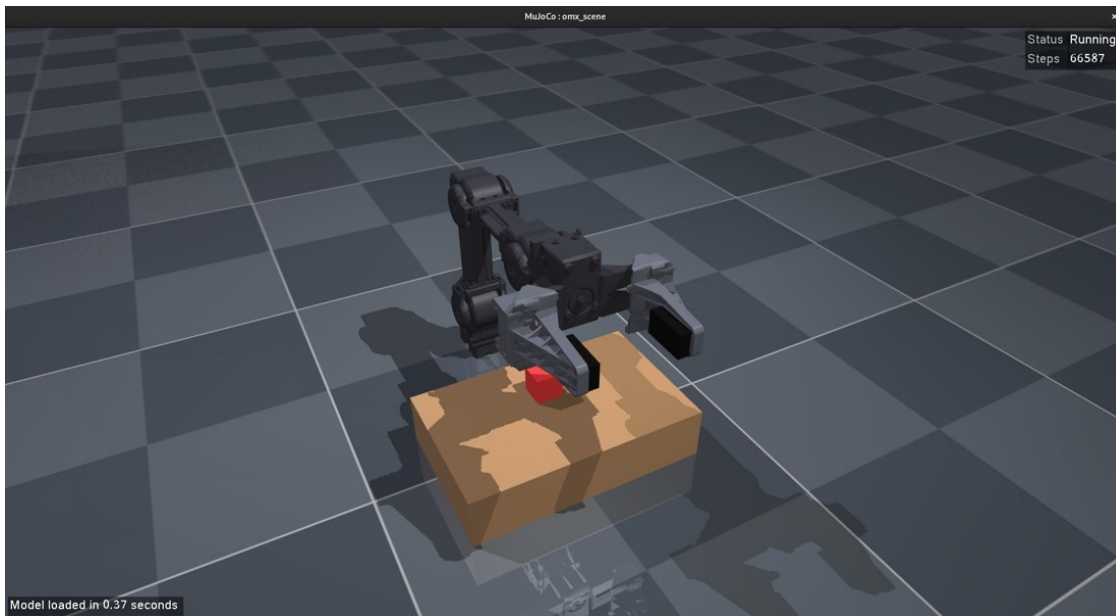


Figure 1: The 4-DOF arm rendered with real meshes, above the table and target cube.

7. TF Tree Verification

The transform tree was generated and inspected using the ROS 2 tf2 tools:

```
ros2 run tf2_tools view_frames
```

The resulting tree, shown below, is a single connected chain in which every joint publishes a valid transform, confirming a correct kinematic structure:

world → link1 → link2 → link3 → link4 → link5 → { gripper_left_link, gripper_right_link, end_effector_link }

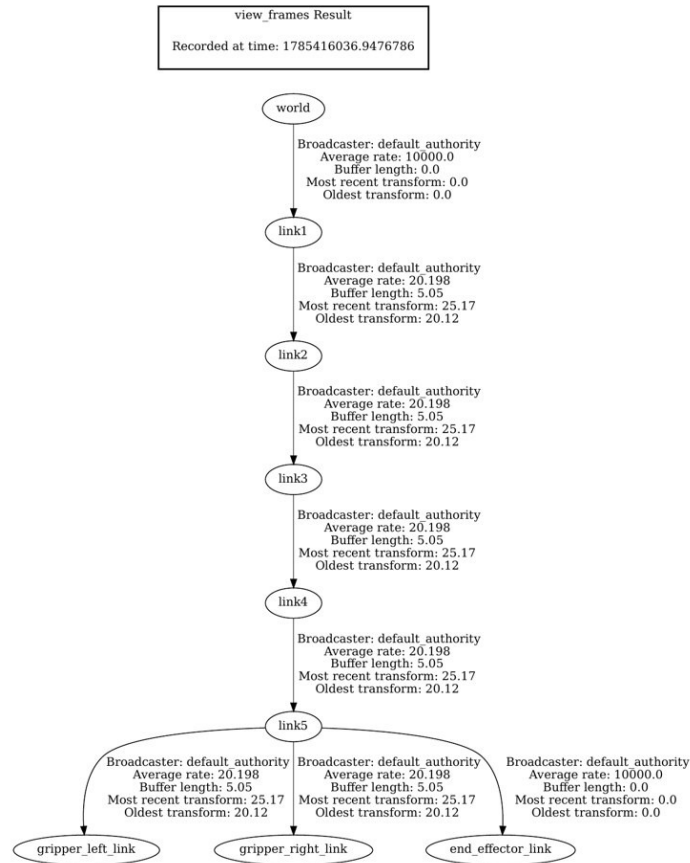


Figure 2: TF tree (view_frames output) — all frames connected from world to the end-effector.

8. Tools Used

- ROS 2 (Jazzy) — robotics middleware and build system.
- URDF / Xacro — robot description format.
- RViz2 — 3D visualization of the robot model.
- robot_state_publisher — publishes the kinematic transforms from the URDF.
- tf2_tools (view_frames) — generates and verifies the TF tree.

9. File Organization

The project is organised as a ROS 2 package with a clear separation of description, meshes and launch files:

```
omx_mujoco/
  urdf/      omx.urdf.xacro      # robot description (links + joints)
```

```

mujoco/assets/ *.stl          # real OpenMANIPULATOR-X meshes
launch/    sim.launch.py      # robot_state_publisher + bringup
config/,   package.xml, CMakeLists.txt

```

10. Results and Evaluation

The project meets every requirement of the problem statement. The results map directly onto the stated evaluation criteria:

Criterion	Outcome
Correctness of implementation	All 5 links and 4 revolute joints (plus gripper) defined with correct axes and limits.
Functional demonstration	Model loads and visualizes; joints move; TF tree is complete and connected.
Code and file organization	Clean ROS 2 package layout (urdf / meshes / launch / config).
Clarity of results and documentation	Geometry, joint limits, TF tree and demonstration documented with figures.

11. Extension: Simulation and Pick-and-Place

As additional work beyond the required scope, the same 4-DOF description was driven in the MuJoCo physics simulator through the `mujoco_ros2_control` bridge, and MoveIt 2 was configured for motion planning. A complete pick-and-place task was implemented in which the arm plans collision-free motions, grasps a cube on a table, lifts it, carries it, and places it — with the grasp handled by simulated contact and friction. The sequence below shows the arm approaching, grasping, and lifting the cube.



Figure 3: Approach — the arm moves in above the target cube.

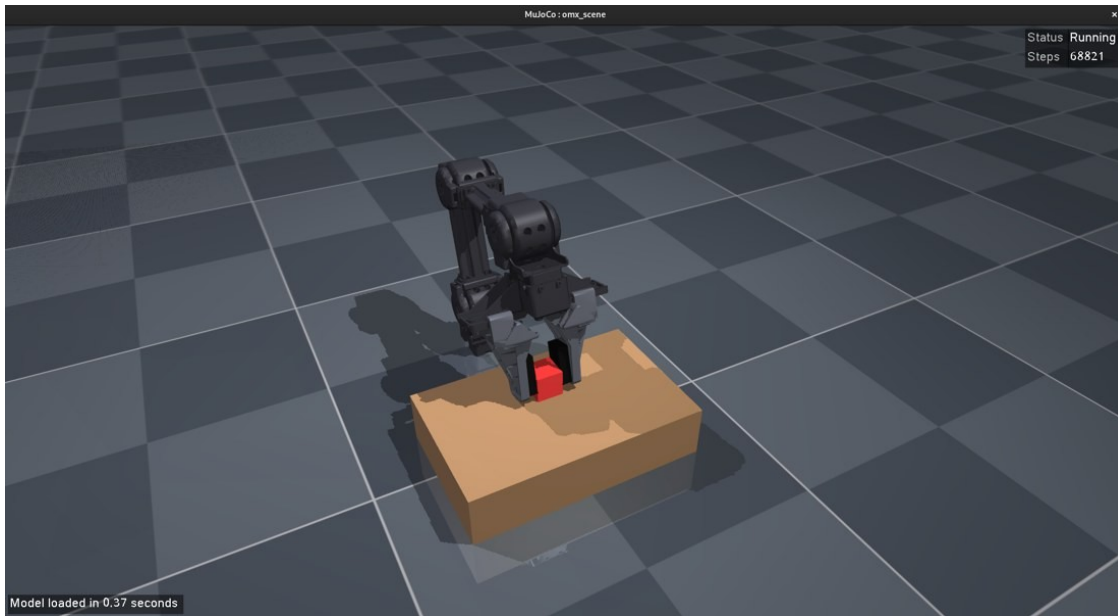


Figure 4: Grasp — the gripper closes on the cube.

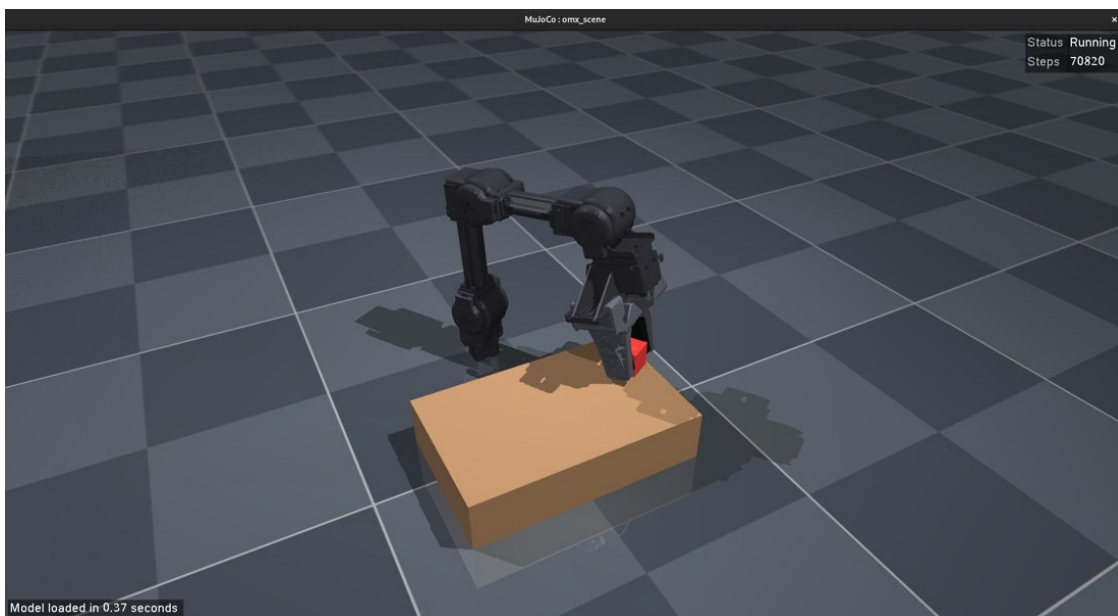


Figure 5: Lift — the cube is held and lifted clear of the table.

11.1 How to Run

The simulation and the pick-and-place task are launched from the ROS 2 workspace in separate terminals. Start the robot first and wait until the arm appears with the table and cube and all three controllers are active, then run the pick-and-place.

```
# Terminal 1 - launch the robot (MuJoCo simulation + controllers)
cd ~/ros2_ws && source install/setup.bash
ros2 launch omx_mujoco sim.launch.py

# Terminal 2 - run the pick-and-place
cd ~/ros2_ws && source install/setup.bash
```

```
ros2 launch omx_moveit_config pick_place.launch.py

# Terminal 3 (optional) - RViz + move_group to watch the planning
cd ~/ros2_ws && source install/setup.bash
ros2 launch omx_moveit_config moveit.launch.py
```

12. Conclusion

A complete and correct ROS 2 URDF model of a 4-DOF industrial robotic arm was designed and validated. The model defines all links and joints with proper visual and collision geometry, uses correct joint types and limits, visualizes correctly, and produces a valid TF tree. The work satisfies all objectives of the problem statement and was further extended to a working MoveIt-driven pick-and-place simulation.